CSC-239
Java Programming:
Chapter 8: Objects and Classes

October 6, 2025

NOTICE to Students: Portions of these lecture slides are
Copyright 2021 Pearson Education, Inc.
We have a license to use this material ***only*** in the context of this course.
There is NO PERMISSION to share these lecture slides with anyone not taking the course.
There is NO PERMISSION to post these lecture slides on the Internet.

Reminder: Quiz 2

- Quiz 2 will begin on Monday, October 6.
  - Online (Moodle) Quiz.
  - The Quiz will open on Monday at 9:00 p.m. and remain open for approximately one week.
  - Each student will have two hours to complete the Quiz, from the time that they begin.
- The material covered on Quiz 2 will be
  - The lectures of September 10 through 29.
  - Chapters 4 through 7.
  - The quiz will include True / False, Multiple-Choice questions.

Two More Reminders:

1. **Project 1** is due on Wednesday, October 15.
2. **Remember** these details (in the ***Syllabus***) about Lab Exercise due dates:
   - Students should submit the source code for their **Lab** solution to *Moodle* as soon as **they** feel it is working correctly.
   - Late penalties are imposed based on the **submit** date, not the date of demonstration during class.
   - The instructor maintains a record of whether each student has ***successfully*** demonstrated the solution to each Lab Exercise. This data becomes a significant part of the student's final course grade.

Online Information

- Java Tutorials:

  https://docs.oracle.com/javase/tutorial/

- Java API:

  https://docs.oracle.com/en/java/javase/21/docs/api/index.html

- The official "*Grader Than*" documentation:

  https://docs.graderthan.com/student/

- *Grader Than* Support E-mail:

  support@graderthan.com

Special NOTE Regarding Quiz 2 (1)

Several questions in **Quiz 2** require that you study the source code of a small Java program, and answer the question: "What is the output of this program?"

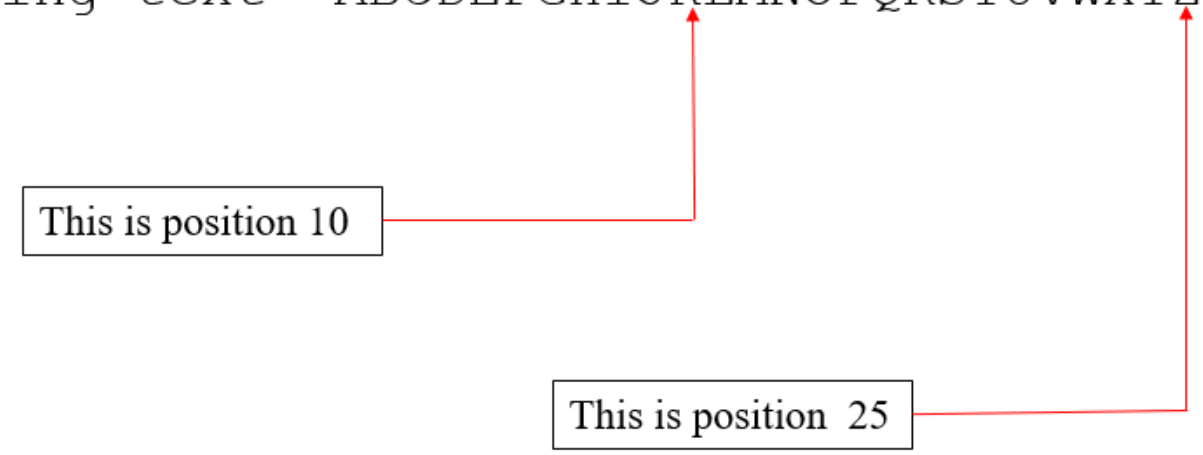
- To answer the question correctly, you need to read the program carefully and *manually* keep track of the program's "current" position in a **String** that is part of the code.
- To make this task easier, the program includes a special comment that helps you to see the exact position of each character in the **String**.
- The next slide contains an example that shows what this special comment looks like:

Special NOTE Regarding Quiz 2 (2)

The '**K**' character is at position 10 in the **String**.

The '**Z**' character is at position 25 in the **String**.

```
// These numbers are intended as a help
// while you are keeping track of the
// position within a String object:
//                         1         2          3
//               0....5....0....5....0....5....0
String text="ABCDEFGHIJKLMNOPQRSTUVWXYZ";
```



OO Programming Concepts

- Object-oriented programming (OOP) involves programming using objects.
- An *object* represents an entity in the real world that can be distinctly identified. For example:
  - A student
  - A course
  - A bank account
- An object has a unique identity, state, and behaviors.
  - The *state* of an object consists of a set of *data fields* (also known as *properties* or *member variables*) with their current values.
  - The *behavior* of an object is defined by a set of methods.

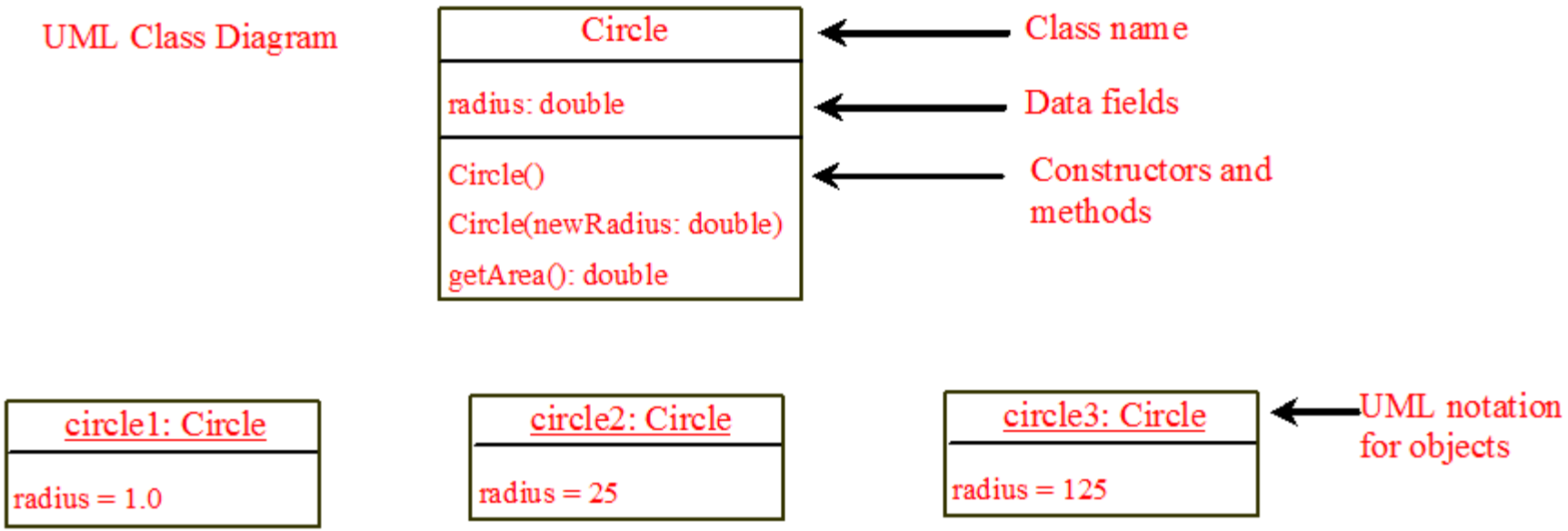
Classes

- *Classes*: constructs that define objects of the same type.
- Words used to describe a class:
  - Template (for creating multiple objects of the same type)
  - Blueprint (like the plans for building a house)
  - Contract (between the class implementer and user)
- A Java class
  - uses member variables to define data fields
  - uses methods to define behaviors
  - Special type of method: constructor, used to construct / initialize an object of the class

UML Class Diagram

Unified Modeling Language (UML):

- Used for modeling software intensive systems.



Declaring Object Reference Variables

To reference an object, assign the object to a reference variable.

To declare a reference variable, use the syntax:

```
ClassName objectRefVar;
```

Example:

```
Circle myCircle;
```

Creating the Object

- Syntax:

```
objectRefVar = new ClassName();
```

- Example:

```
myCircle = new Circle();
```

- This is also called “instantiation” or creating an instance of the class.

  (They seem to have a name for everything!)

Declaring/Creating Objects in a Single Step

ClassName *objectRefVar* =

new *ClassName*();

Example:

Circle myCircle = new Circle();

Constructors

- Constructors are a special kind of method, automatically invoked to initialize an object.
- A class can have many constructors:

```
Circle()   {     }


Circle(double newRadius) {
     radius = newRadius;
}
```

Constructors, continued

- A constructor with no parameters: referred to as a *no-arg constructor*.
- Constructors must have the same name as the class itself.
- Constructors do not have a return type—not even void.
- Constructors are invoked using the **`new`** operator when an object is created.
- Constructors play the role of initializing objects.

Creating Objects Using Constructors

new *ClassName*();

Example:

new Circle();

new Circle(5.0);

Default Constructor

- A class may be defined without constructors.
- In this case, a no-arg constructor with an empty body is implicitly declared in the class.
- This constructor, called *a default constructor*, is provided automatically *only if no constructors are explicitly defined in the class*.

Creating a no-arg Constructor

- If your class has one or more explicit constructors, but it also needs a “no-arg” constructor, then you need to write it yourself:

```
Circle()   {
        //(empty is OK)
  }
```

Accessing Object's Members

- Referencing the object's data:

  objectRefVar.dataFieldName

  For example: myCircle.radius

- Invoking the object's method:

  objectRefVar.methodName(*arguments*)

  For example: myCircle.getArea()

Reference Data Fields

The data fields can be of primitive types or reference types.

For example, the following Student class contains a data field name of the String type.

```
public class Student {
  String name;      // (default value = null)
  int age;          // (default value = 0)
  boolean isScienceMajor; // (default=false)
  char gender;      // (default = '\u0000')
}
```

The null Value

If a data field of a reference type does not reference any object, the data field holds a special literal value, null.

Default Value for a Data Field

The default value of a data field is null for a reference type, 0 for a numeric type, false for a boolean type, and '\u0000' for a char type.

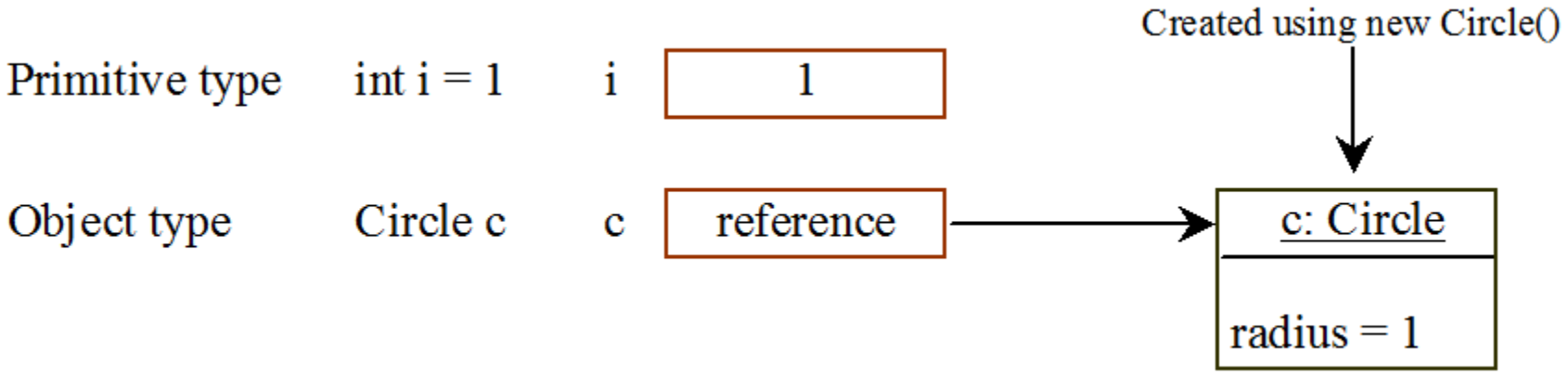
```
public class Test {
 public static void main(String[] args) {
   Student student = new Student();
   System.out.println("name? " + student.name);
   System.out.println("age? " + student.age);
   System.out.println("isScienceMajor? " +
                      student.isScienceMajor);
   System.out.println("gender? "+student.gender);
 }
}
```

Local Variables – no default value

- Java assigns no default value to a local variable inside a method.
- The code below would cause a compilation error: variables not initialized.

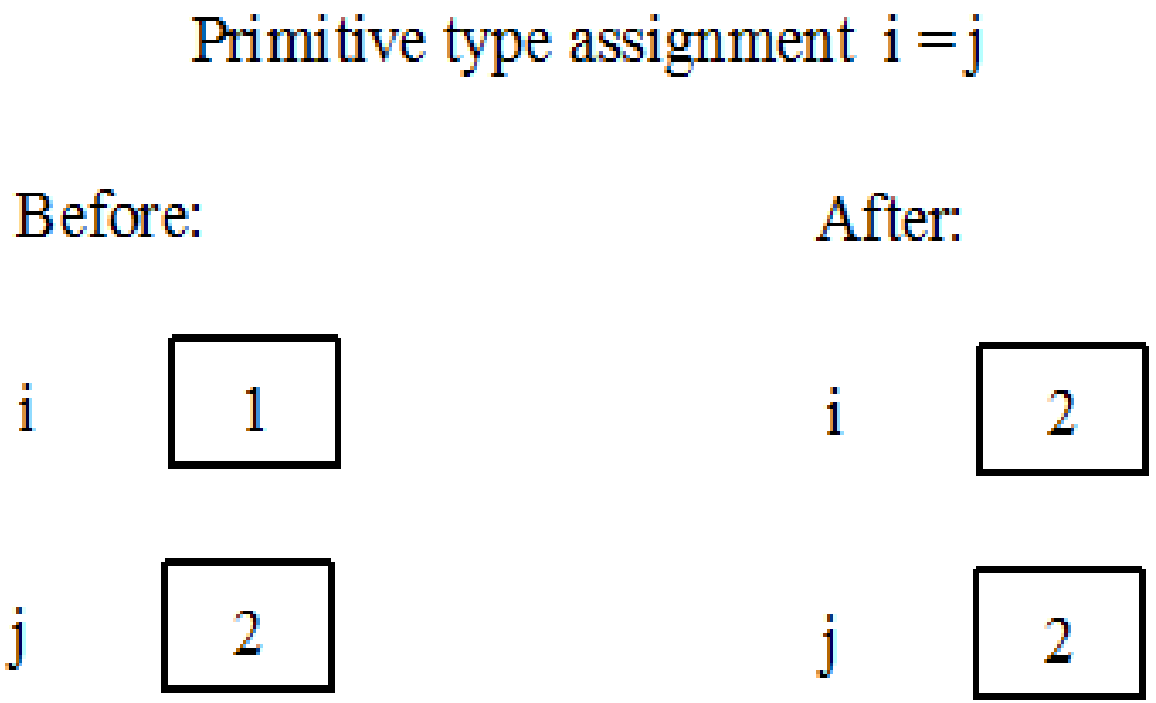
```
public class Test {
  public static void main(String[] args) {
    int x; // x has no default value
    String y; // y has no default value
    System.out.println("x is " + x);
    System.out.println("y is " + y);
  }
}
```

Differences between Variables of Primitive Data Types and Object Types



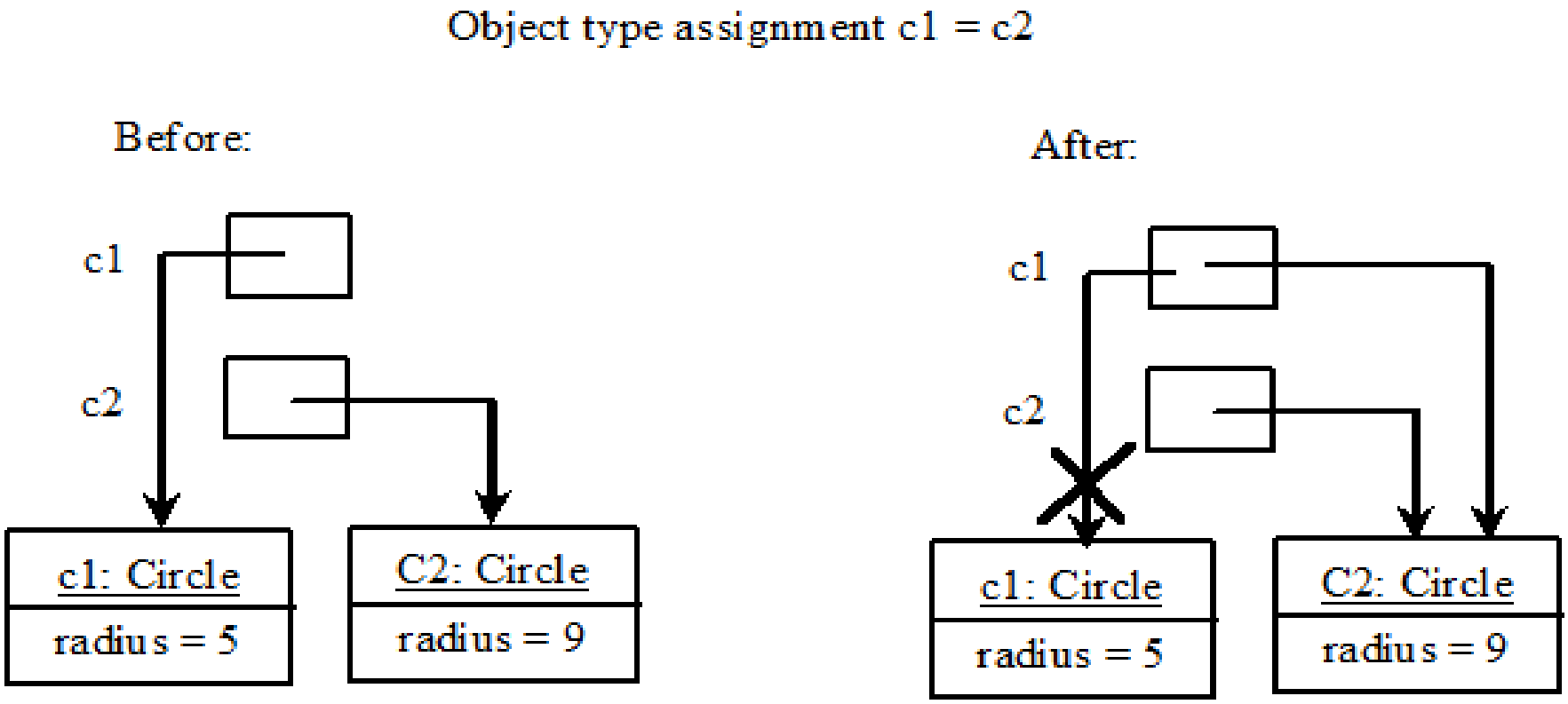
Copying Variables of Primitive Data Types

- Primitive types: copies the actual data



Copying Variables of Object Types

- Object types: copies the object *reference*



Garbage Collection

As shown in the previous figure, after the assignment statement c1 = c2, c1 points to the same object referenced by c2. The object previously referenced by c1 is no longer referenced. This object is known as "garbage". Garbage is automatically collected by the Java Virtual Machine (JVM).

Instance Variables, and Methods

- In the Circle class example, each Circle object has its own **`radius`** variable.
  - The **`radius`** variable is called an *instance variable*, because each instance of the class has its own value for **`radius`**.
- The **`getArea()`** method calculates the area based on the value of **`radius`** for that particular instance. The **`getArea()`** method is called an instance method.
  - An instance method can only be invoked by a particular instance of the class.

Static Variables, Constants, and Methods

- Static variables are shared by all the instances of the class.
- Static methods are not tied to a specific object.
- Static constants are final variables shared by all the instances of the class.
- To declare static variables, constants, and methods, use the **`static`** modifier.

Visibility Modifiers

- By default, the class, variable, or method can be accessed by any class in the same package.
- Specific Visability Modifiers:

`public`	The class, data, or method is visible to any class in any package.
`private`	The data or methods can be accessed only by the declaring class.

Accessor/Mutator Methods

- If a data member is “private”, how can code outside the class read or write the data?
- The get and set methods (usually `public`) are used to read and modify private properties.
- For each private data field, get***FieldName***() and set***FieldName***() method is written in the class, for the purposing of reading / writing the data field.
- This is known as “Data Field Encapsulation”
  - See also section 9.9, pages 346 – 349.

CircleWithPrivateDataFields (1)

```
public class CircleWithPrivateDataFields {
 private double radius = 1;
 private static int numberOfObjects = 0;
 public CircleWithPrivateDataFields() {
    numberOfObjects++;
 }
 public CircleWithPrivateDataFields(double r){
    radius = r;
    numberOfObjects++;
 }
```

CircleWithPrivateDataFields (2)

```
 public double getRadius() {
   return radius; }
 public void setRadius(double newRadius){
   radius=(newRadius>=0) ? newRadius : 0;
 }
 public static int getNumberOfObjects(){
    return numberOfObjects;
 }
 public double getArea() {
    return radius * radius * Math.PI;
 }
} //(end 'CircleWithPrivateDataFields')
```

TestCircleWithPrivateDataFields

```
public class TestCircleWithPrivateDataFields{
  public static void main(String[] args) {
  CircleWithPrivateDataFields myCircle =
      new CircleWithPrivateDataFields(5.0);
  System.out.println("radius="
      + myCircle.getRadius()
      + " Area=" + myCircle.getArea());

  myCircle.setRadius(myCircle.getRadius()
       * 1.1); // (increase by 10%)
    System.out.println("radius="
      + myCircle.getRadius()
      + " Area=" + myCircle.getArea());
  }
}//(end 'TestCircleWithPrivateDataFields')
```

Why Make Data Fields private?

- *Most often*, it makes sense to make data members private. (There are exceptions, of course.)
- To protect data (from code outside the class).
- To make class easier to maintain:
  - If the implementation details need to change, then any code which uses the class should not need to change.
  - If the class developer makes particular implementation details private, then he / she retains the option to change those details in the future, without altering the "contract" of the class.

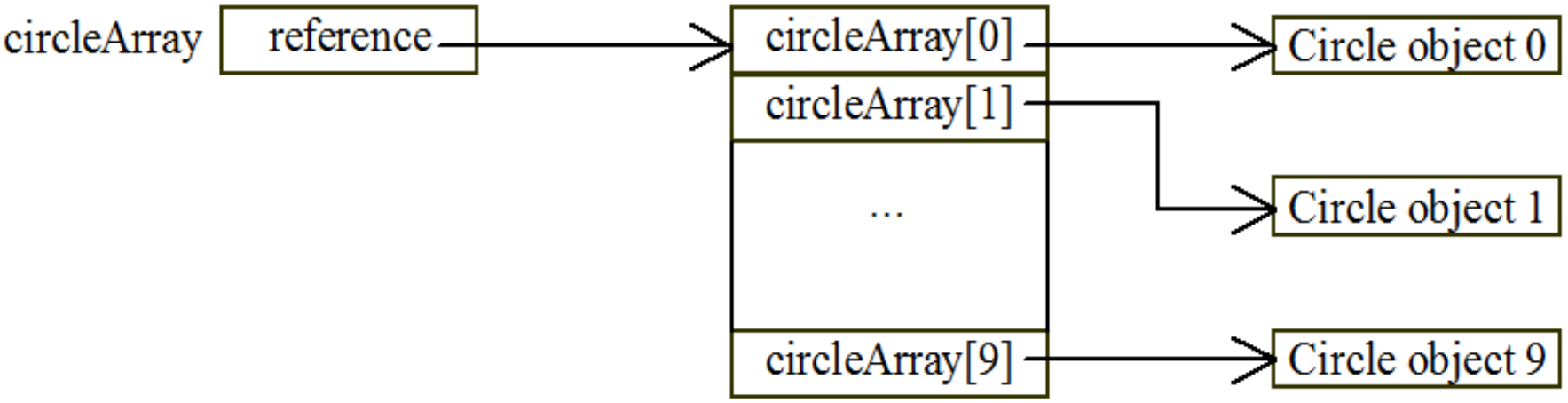
Passing Objects to Methods

- When an argument of a primitive type is passed to a method, it is passed *by value*.
  - The *value* is passed to the parameter inside the method.
- When an object is passed to a method, it is the *value of the reference variable* which is passed to the method.
  - This is effectively "passing by reference".

Array of Objects

```
Circle[] circleArray = new Circle[10];
```

- An array of objects is actually an *array of reference variables*.
- `circleArray` references to the entire array.
- `circleArray[1]` references to a Circle object.



Example: a class to represent rectangles.

- Create a class called "Rectangle":
  - Two `double` data fields: `width` and `height`
  - Two constructors (one with no-args)
  - Two methods: `getArea()`, `getPerimiter()`
- Create a test program that creates two Rectangle objects:
  - width = 4, height = 40
  - width = 3.5, height = 35.9
- The test program should display width, height, area and perimeter of each rectangle.

Designing the Rectangle Class

- Two data fields, each with a default value:

```
double width = 1, height = 1;
```

- A *no-arg* constructor:

```
public Rectangle() {        }
```

- A constructor that accepts parameters:

```
public Rectangle(double w, double h) {
   width = w;
   height = h;
}
```

“Getter” and “Setter” Methods

```
public double getWidth() {
     return width;
}
public double getHeight() {
     return height;
}
public void setWidth(double w) {
     width=w;
}
public void setHeight(double h) {
     height=h;
}
```

Rectangle Methods

```
public double getArea() {
    return width * height;
}

public double getPerimiter() {
    return 2*(width + height);
}
```

Rectangle Test Program

```
public class TestRectangle {
    public static void main(String[] args) {
        Rectangle myRectangle = new Rectangle(4, 40);
        System.out.printf("width=%6.2f, height=%6.2f\n",
                myRectangle.width, myRectangle.height);
        System.out.printf("perimiter=%6.2f, area=%6.2f\n",
                myRectangle.getPerimeter(),
                myRectangle.getArea());

        Rectangle yourRectangle = new Rectangle(3.5, 35.9);
        System.out.printf("width=%6.2f, height=%6.2f\n",
                yourRectangle.width, yourRectangle.height);
        System.out.printf("perimiter=%6.2f, area=%6.2f\n",
                yourRectangle.getPerimeter(),
                yourRectangle.getArea());
    }
} //(end class TestRectangle)
```

Immutable Objects and Classes

- Immutable object: cannot be changed once the object is created.
  - For example: objects of the String class
- Immutable class: generates an immutable object.
  - All data fields must be private.
  - No mutator ("setter") methods.
  - No accessor ("getter") methods that return a reference to a mutable data field object.

Example: Student *Appears* Immutable

```
public class Student {
  private int id;
  private BirthDate birthDate;

  public Student(int ssn,
      int year, int month, int day) {
    id = ssn;
    birthDate = new BirthDate(year, month, day);
  }

  public int getId() {
    return id;
  }

  public BirthDate getBirthDate() {
    return birthDate;
  }
}
```

`BirthDate`: NOT an Immutable Class

```
public class BirthDate {
  private int year;
  private int month;
  private int day;

  public BirthDate(int newYear,
      int newMonth, int newDay) {
    year = newYear;
    month = newMonth;
    day = newDay;
  }

  public void setYear(int newYear) {
    year = newYear;
  }
}
```

- In the BirthDate class, the setYear() method is a mutator method.

`Student` -- Not immutable after all

```
public class Test {
  public static void main(String[] args) {
    Student student =
        new Student(111223333, 1970, 5, 3);
    BirthDate date = student.getBirthDate();
    date.setYear(2010);
   // Now the student birth year is changed!
  }
}
```

- The Student.getBirthDate() method returns a reference to an object, birthDate, which itself is mutable.
- Therefore, the Student class is really NOT immutable.

How to fix Student / Birthday Classes?

- Think about the BirthDate class:
  - Why does it need a setYear() method?
  - In fact, a setYear method makes no real sense in this class, because none of us can change the year we were born! (Even if we wanted to.)
- If you throw out the setYear() method, then BirthDate becomes an immutable class.
- Then, the Student class (as defined in this example) is also an immutable class.

Scope of Variables

- The scope of instance and static variables is the entire class. They can be declared anywhere inside a class.
- The scope of a *local* variable (inside a method) starts from its declaration and continues to the end of the block that contains the variable.
- A local variable must be initialized explicitly before it can be used.

The `this` Keyword

- The **`this`** keyword is the name of a reference that refers to the "current" object itself.
  - One common use of the **`this`** keyword is to reference a class's *hidden data fields*.
  - Another common use of the **`this`** keyword is to enable one constructor to invoke another constructor of the *same* class.

Reference the Hidden Data Fields

```
public class F {
  private int i = 5;
  private static double k = 0;

  void setI(int i) {
    this.i = i;
  }

  static void setK(double k) {
    F.k = k;
  }
}
```

```
Suppose that f1 and f2 are two objects of F.
F f1 = new F(); F f2 = new F();

Invoking f1.setI(10) is to execute
   this.i = 10, where this refers f1

Invoking f2.setI(45) is to execute
   this.i = 45, where this refers f2
```

- Inside method `setI()`, the object's `i` instance variable would normally be hidden by the `i` calling parameter.
- Using the `this` keyword provides a way to access the instance variable.

Lab09a: **Stock** class

- Create a class called **Stock** that represents a particular stock that you could buy or sell on the stock market.
- **Stock** class member variables:

```
String symbol;
String name;
double previousClosingPrice;
double currentPrice;
```

- In addition to constructors, getters and setters (as needed), the **Stock** class must have a method that calculates how much the stock price has changed: **getChangePercent**.